An open, containerized pipeline turning multi-image captures — from consumer 360° cameras to terrestrial laser scanners — into web-navigable 3D scenes

Author: Angel Naya, Independent Researcher, Berlin, Germany
Corresponding author: Angel Naya, angel.naya@posteo.de

Abstract

Walkable 360° panoramas and point clouds are widely used for documentation, monitoring, and outreach, but producing them usually relies on proprietary, cloud-bound tools. We present an open-source, containerized processing chain with a browser-based graphical user interface that turns multi-image captures into walkable web scenes. The chain handles two input classes that are normally treated separately: captures without known pose (consumer 360° cameras, DSLR with fisheye lens), which are stitched with Hugin, and the RGB images of terrestrial laser scanners, whose poses are stored in the E57 file and which are therefore reprojected directly. The input class is detected automatically. Scenes are served by an open-source web viewer (Pannellum, three.js) and can be self-hosted (Docker, Caddy, Garage) or exported as a static site. In an evaluation against eleven CC0 reference panoramas, the pose-based branch outperforms stitching on every measure (26.9 vs 23.0 dB PSNR, 0.886 vs 0.837 SSIM, 0.10 vs 0.14 px seam offset), yields a usable panorama for all eleven scenes versus six, and is about seven times faster. Software, container recipe, and evaluation code are openly available; all example data are third-party open data.

Keywords: panorama stitching; 360° imaging; terrestrial laser scanning; E57; open-source; self-hosting; Docker; web visualization

Highlights

- Open, containerized GUI chain turns 360° and TLS captures into web 3D scenes
- The input class is detected automatically: stitch, reproject, or take over
- Pose-based reprojection of E57 scanner images beats stitching on every metric
- Reprojection gives usable panoramas for 11 of 11 scenes, stitching for 6
- Real parallax raises seam offset ~56× over a parallax-free synthetic control

1. Introduction

Interactive 360° panoramas and 3D point clouds are used to document and communicate spatial states in forest and environmental monitoring, surveying, cultural heritage, and teaching. Two obstacles recur. First, the common routes are either proprietary and cloud-bound

(camera apps with vendor hosting) or fragmented: a stitching tool, a web viewer, and a server must be wired together by hand, and there is no reproducible path from raw captures to a published, walkable scene. Second, a valuable data source is routinely left unused: terrestrial laser scanners (TLS) record RGB images whose camera poses are stored alongside the geometry in the E57 exchange format [5], yet these images serve mainly to colourize the point cloud or, as panoramic reflectance images, to register scans [6, 7] — not as navigable panoramas in their own right.

We address both with one open-source tool. Its contribution is not an individual component — panorama stitching [1, 2] and web panorama viewers [3] are established — but their integration into **one automated, containerized, GUI-driven chain**, and specifically the **unification of the two input classes**: captures whose pose must be estimated are stitched; captures whose pose is known are reprojected directly to the equirectangular domain, which is faster and free of stitching failure modes. Scenes can carry georeferenced markers, for example tree-inventory data derived from the same point cloud, and are served fully self-hosted, without a cloud dependency. The same chain thus covers the device spectrum from consumer 360° cameras to survey-grade TLS.

2. Software design

2.1 Software architecture

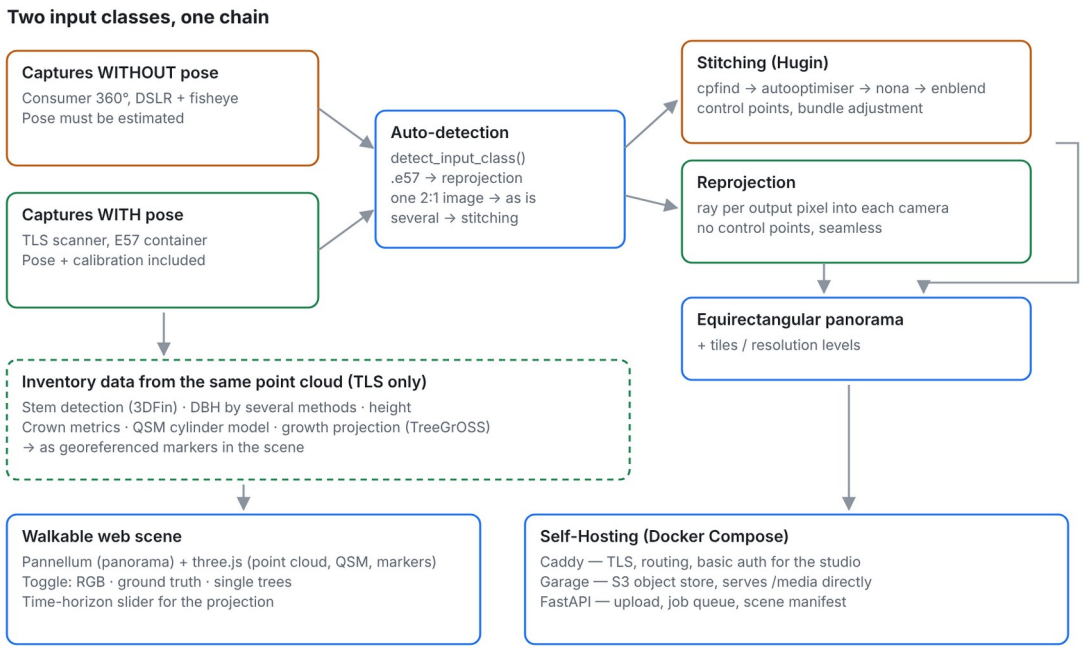


Figure 1. *Two input classes, one chain. The input class is determined from the data (`detect_input_class`), not declared by the user: an `.e57` container carries pose and is reprojected; a single 2:1 image is taken over as is; several images without pose are stitched. For TLS input, the point cloud is added to the scene; inventory data derived from it with the repository scripts can be attached as georeferenced markers (dashed).*

The system is packaged as Docker services orchestrated by Compose: a processing/API service (Python, FastAPI, job queue), an S3-compatible object store (Garage), a reverse proxy with automatic TLS (Caddy), and the browser-based studio GUI. A single command brings up the stack.

At upload, the input class is determined automatically (`detect_input_class`): an `.e57` file triggers reprojection; a single image with a 2:1 aspect ratio is accepted as a finished equirectangular panorama; several images trigger stitching. A single image with another aspect ratio is rejected rather than guessed.

2.2 Functionality

- **Stitching branch** (pose unknown): Hugin/Panotools with automatic control-point detection (`cpfind`, `cpclean`), optimization (`autooptimiser`), and seam blending (`nona`, `enblend`); no manual control points.
- **Reprojection branch** (pose known): the per-image poses from the E57 file are used to resample the RGB images into a seamless equirectangular panorama (bilinear sampling).
- **Point cloud**: the E57 point cloud is converted into web-ready levels of detail.
- **Viewing**: panorama viewer (Pannellum [3]) and point-cloud viewer (three.js); for very large clouds, dedicated streaming renderers such as Potree [4] remain the reference.
- **Markers**: placed by clicking into the panorama in the studio GUI, or imported from point-cloud analyses.
- **Publishing**: self-hosted (Caddy + Garage) or as a static export.

3. Application and evaluation

3.1 Scanner RGB → walkable scene with inventory markers

A Leica BLK360 scan of the Renon forest plot (ICOS station IT-Ren, E57 [15]) with six pinhole images and poses per setup is reprojected to a seamless equirectangular panorama; the co-registered point cloud is added, and tree-inventory markers computed with the repository scripts (stem position, DBH, height, crown metrics, growth projection) are placed in the walkable scene. One acquisition thus yields both a visual record and a data carrier.

The markers are only as reliable as the scan geometry behind them. For plot-wide TLS clouds the inventory scripts use 3DFin [11] as stem detector. On two manually labelled plots of the SegmentedForests dataset [12] it reaches 98.5 % recall / 95.7 % precision (plot_06, 68 stems) and 94.5 % / 100 % (plot_07, 128 stems). This accuracy depends on circumferential stem coverage: 3DFin accepts a diameter only if the stem cross-section at breast height is covered in at least 9 of 16 sectors (≈ 202°). The labelled plots offer a median arc of 280–360°; on the Renon stand, merged from four nearby scan positions, the median is 180°, and 3DFin declines a diameter for 29 of 37 detected stems — correct behaviour, since the data do not support one. A systematic comparison of detectors as a function of coverage is beyond the scope of this paper; per-method results are documented in the repository (`scripts/BENCH_DETECTION.md`).

3.2 Panorama quality against CC0 ground truth

From each of eleven CC0 reference panoramas (Poly Haven, 8192×4096) we render synthetic captures and rebuild a panorama (2048×1024) — once by stitching six 180° fisheye frames (pose withheld), once by reprojecting six 90° pinhole frames plus zenith and nadir (pose used) — and measure both against the same original (PSNR, SSIM [8]). A run counts as **completed** if the chain writes a panorama and as **usable** if the result is also geometrically correct.

Branch	completed	usable	PSNR (dB)	SSIM	seam offset (px)	runtime
Stitching (pose estimated)	8 / 11	**6 / 11**	22.98 ± 2.74	0.837 ± 0.060	0.14 (p95 0.53)	14.6 s
Reprojection (pose known)	11 / 11	**11 / 11**	**26.87 ± 1.96**	**0.886 ± 0.039**	**0.10 (p95 0.23)**	2.1 s

Aggregate figures refer to the usable runs. The pose-based branch is better on every measure (+3.9 dB, +0.05 SSIM, a third less local displacement) and reconstructs all eleven scenes. Stitching fails on five: three times `enblend` aborts (excessive overlap, degenerate mask geometry, no detectable seam), and twice the result is geometrically wrong (8.6 and 11.7 dB, confirmed by exhaustive search over rotation, tilt, and mirroring).

Seam offset is the local displacement measured by block-wise phase correlation [9] (64-px blocks, 50 % overlap, low-structure blocks discarded). Against the reference panorama it separates good from failed reconstructions more sharply than PSNR or SSIM: medians of 0.07–0.13 px (reprojection) and 0.11–0.19 px (good stitches) versus 5.95–6.14 px (failed stitches), a factor of ~40.

The synthetic captures share one nodal point and simulate no sensor noise, exposure differences, or lens distortion; they measure the geometry of the chain, not camera image quality. Absolute PSNR is limited by the resolution chain and is meaningful only in the comparison of the two branches.

3.3 Real captures: the cost of parallax

To quantify how far the synthetic setting is from reality, `eval_seams.py` measures seam offset **without a reference panorama**: `nona` remaps each capture separately into the panorama canvas, and where two source images overlap they show the same viewing direction — with perfect registration and rotation about the nodal point they would be identical there. Their local displacement is the error.

The real capture is **PASSTA LunchRoom** [10]: 72 photographs from a rotating Canon EOS 70D with supplied calibration. As a control, synthetic captures with identical geometry are rendered from a CC0 panorama (18 positions, 93.6° rectilinear, same stitching chain, same canvas width), so that the real captures differ only by genuine parallax, sensor noise, and exposure differences:

	synthetic (parallax-free)	real (rotating camera)
seam offset, median	**0.07 px**	**3.95 px**
p95	0.18 px	24.78 px
maximum	0.48 px	29.99 px
blocks above 1 px	0.0 %	67.5 %

The median is ~56× higher and the 95th percentile ~140×. The synthetic stitching figures are therefore upper bounds of real stitching quality, and the advantage of reprojection measured in Section 3.2 is a lower bound. PASSTA LunchRoom is an indoor scene with nearby objects, where parallax is strongest; outdoor scenes with mostly distant objects will fare better. The reprojection branch is unaffected: it has no overlap seams, because every viewing direction comes from exactly one camera. For an automatic quality flag this means that a threshold derived from synthetic data (e.g. > 1 px) does not transfer to real captures and has to be calibrated on real data.

3.4 Runtime per input class

Measured end to end through the running containerized service (upload, queue, processing, publication), with the input type not declared:

Input class	captures	input	upload	processing	**total**
Consumer 360° (finished equirect)	1	2.5 MB	0.2 s	2.0 s	**2.2 s**
DSLR/fisheye (stitching)	18	6.9 MB	0.4 s	69.3 s	**69.7 s**
TLS scan (E57, reprojection)	1	177.9 MB	4.8 s	27.7 s	**32.5 s**

Medians of two runs each on a consumer CPU; the input class was detected correctly in all runs, and no manual intervention was needed. The 178 MB scan finishes in 32 s, the 6.9 MB of individual frames take 70 s: cost comes from registration, not from data volume — precisely what pose-based reprojection avoids.

4. Discussion

The tool makes reproducible, self-hosted 360°/3D documentation available without vendor lock-in, from consumer cameras to TLS, through one GUI. It turns the pose-bearing RGB images of laser scanners into navigable panoramas; the evaluation shows why this is preferable whenever pose is available (no stitching failures, higher accuracy, ~7× faster). For forest and environmental monitoring, one acquisition yields a walkable panorama, the point cloud, and inventory markers in the same scene, which supports communication and revisiting of plots. Seam offset provides a candidate quality signal for automated batch processing, subject to calibration on real captures.

5. Conclusions

We presented an open-source, containerized, GUI-driven chain that converts multi-image captures — with or without known pose — into walkable web scenes, self-hostable end to end. A ground-truth evaluation shows the pose-based reprojection branch to be more accurate, more robust, and faster than stitching when scanner poses are available; a reference-free measurement on real captures quantifies how strongly parallax degrades stitching. Future work: calibrate the seam-offset quality flag on real captures and integrate it into the GUI; add a freely licensed set of raw dual-fisheye frames from a consumer 360° camera to the evaluation; and run a usability study with non-expert users (System Usability Scale [13, 14]).

Code availability

- Name of the software: For3Dsuite
- Developer and contact: Angel Naya, angel.naya@posteo.de
- Year first available: 2026
- Hardware requirements: consumer PC; no GPU required (runtimes in Section 3.4)
- Software requirements: Docker and Docker Compose
- Program languages: Python, JavaScript
- Program size: about 2 MB of source code
- License: GNU General Public License v3.0 or later (GPL-3.0-or-later)
- Source code: https://github.com/anayana/For3Dsuite
- Archived version (v1.0): https://doi.org/10.5281/zenodo.23323560

Data availability

All data used in this work are third-party open data. Terrestrial laser scans: Renon / ICOS IT-Ren (Leica BLK360, E57; CC BY 4.0) [15] and SegmentedForests (MIT) [12]. Reference panoramas: eleven Poly Haven HDRIs (CC0, https://polyhaven.com). Real multi-image capture: PASSTA LunchRoom (CC BY 4.0) [10]. Source code, container recipe, and evaluation scripts: https://github.com/anayana/For3Dsuite (GPL-3.0-or-later), archived at https://doi.org/10.5281/zenodo.23323560.

Declaration of competing interest

The author declares no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

CRediT authorship contribution statement

Angel Naya: Conceptualization, Methodology, Software, Validation, Formal analysis, Data curation, Visualization, Writing – original draft, Writing – review & editing.

Acknowledgements

The author thanks the providers of the open datasets used here: Chang et al. and the ICOS ecosystem station IT-Ren for the Renon laser scans, the authors of SegmentedForests and

PASSTA, and the Poly Haven contributors. This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

References

[1] Brown, M., Lowe, D. G. (2007). Automatic panoramic image stitching using invariant features. *International Journal of Computer Vision* 74(1), 59–73. https://doi.org/10.1007/s11263-006-0002-3

[2] Szeliski, R. (2006). Image alignment and stitching: a tutorial. *Foundations and Trends in Computer Graphics and Vision* 2(1), 1–104. https://doi.org/10.1561/0600000009

[3] Petroff, M. A. (2019). Pannellum: a lightweight web-based panorama viewer. *Journal of Open Source Software* 4(40), 1628. https://doi.org/10.21105/joss.01628

[4] Schütz, M. (2016). *Potree: Rendering large point clouds in web browsers.* Diploma thesis, TU Wien.

[5] Huber, D. (2011). The ASTM E57 file format for 3D imaging data exchange. *Proc. SPIE* 7864, Three-Dimensional Imaging, Interaction, and Measurement, 78640A. https://doi.org/10.1117/12.876555

[6] Houshiar, H., Elseberg, J., Borrmann, D., Nüchter, A. (2015). A study of projections for key point based registration of panoramic terrestrial 3D laser scan. *Geo-spatial Information Science* 18(1), 11–31. https://doi.org/10.1080/10095020.2015.1017913

[7] Kang, Z., et al. (2009). Automatic registration of terrestrial laser scanning point clouds using panoramic reflectance images. *Sensors* 9(4), 2621–2646. https://doi.org/10.3390/s90402621

[8] Wang, Z., Bovik, A. C., Sheikh, H. R., Simoncelli, E. P. (2004). Image quality assessment: from error visibility to structural similarity. *IEEE Transactions on Image Processing* 13(4), 600–612. https://doi.org/10.1109/TIP.2003.819861

[9] Kuglin, C. D., Hines, D. C. (1975). The phase correlation image alignment method. *Proc. IEEE Int. Conf. on Cybernetics and Society*, 163–165.

[10] Meneghetti, G., Danelljan, M., Felsberg, M. (2015). Image alignment for panorama stitching in sparsely structured environments [Dataset]. Zenodo. CC BY 4.0. https://doi.org/10.5281/zenodo.19663081

[11] Laino, D., Cabo, C., Prendes, C., et al. (2024). 3DFin: a software for automated 3D forest inventories from terrestrial point clouds. *Forestry* 97(4), 479–496. https://doi.org/10.1093/forestry/cpae020

[12] Laino, D., Cabo, C., Ordóñez, C., et al. (2026). SegmentedForests: a labelled dataset of terrestrial LiDAR point clouds for semantic segmentation of forests. *Forestry* 99(2). https://doi.org/10.1093/forestry/cpaf062 · Data: https://doi.org/10.5281/zenodo.17396681

[13] Brooke, J. (1996). SUS: a “quick and dirty” usability scale. In: *Usability Evaluation in Industry*, Taylor & Francis, 189–194.

[14] Sauro, J., Lewis, J. R. (2016). *Quantifying the User Experience: Practical Statistics for User Research*, 2nd ed., Morgan Kaufmann.

[15] Chang, M., Candotti, A., von Ellenrieder, K., Tomelleri, E., Camurri, M. (2025). Data for: “A multi-platform LiDAR dataset for standardized forest inventory measurement at long term ecological monitoring sites” [Dataset]. Zenodo. https://doi.org/10.5281/zenodo.17186174

Software used, with the versions that produced the reported figures: Hugin/Panotools 2024.0.1 (GPL-2.0), Pannellum 2.5.6 (MIT), three.js r160 (MIT), Caddy 2 (Apache-2.0), Garage 1.0.1 (AGPL-3.0), FastAPI (MIT), NumPy 1.26.4, Pillow 12.3.0, OpenCV 5.0.0, laspy 2.5.4, 3DFin/dendromatics, TreeGrOSS (GPL-3.0, isolated as a separate process).